

Six compromises, worked end to end

The same frame applied six times – and the two skills that make each one answerable.

The frame never changes. **Validate, contain, capture, eradicate, recover.** Only the containment action differs.

Validating a finding: scope and impact

- ◆ **New in SCS-C03, skill 2.2.3.** A finding tells you something happened. It does not tell you how much.
- ◆ **Is it real?** Approved penetration test, known scanner, expected automation. Check before you page anybody.
- ◆ **What identity?** Resolve the role or user, then find every session and every other resource it could reach.
- ◆ **What else did it touch?** CloudTrail for the principal, not just the resource named in the finding.
- ◆ **Severity is not impact.** A HIGH finding on a sandbox instance matters less than a MEDIUM on a payments role.

ON THE EXAM

The finding names one resource. The incident is the principal.

Amazon Detective and root cause

- ◆ Builds a **behavior graph** from CloudTrail, Flow Logs, EKS audit logs and GuardDuty and Security Hub findings.
- ◆ **Finding groups** correlate related findings and entities into one incident.
- ◆ **Detective Investigation** analyses an IAM user or role for indicators of compromise.
- ◆ **Up to one year** of data. **Regional**. Needs about **two weeks** to build baselines.
- ◆ Only the **delegated administrator** sees the graph. Members contribute and see nothing.

ON THE EXAM

Detective is investigation, not detection. It collects no artefacts.

A developer commits a long-lived IAM user access key to a public GitHub repository. Within nine minutes, CloudTrail shows the key calling sts:AssumeRole against a data-processing role, and that role reading from three S3 buckets.

The responder immediately deactivates the exposed access key. Twenty minutes later the role is still being used to list and read objects from the same buckets.

What should the responder do next?

- ☐ A Delete the IAM user entirely, which terminates all sessions derived from its credentials
- ☐ B Rotate the S3 bucket encryption keys so the attacker's session cannot decrypt objects
- ☐ C Attach a Deny conditioned on aws:TokenIssueTime to the assumed role
- ☐ D Remove the bucket policies granting the role access, then restore them after the incident

◆ PAUSE AND DECIDE

What should the responder do next?

A Delete the IAM user to terminate derived sessions

The key misconception from module 10. Role session credentials are independent of the user that created them and survive the user's deletion.

C Deny on aws:TokenIssueTime on the assumed role

Correct. It denies every session issued before now, including the attacker's, without disturbing legitimate consumers who assume the role afterwards.

B Rotate the bucket encryption keys

SSE-KMS decryption is authorised through the key policy and the caller's permissions, not a key version the session holds. Rotation changes nothing for this session.

D Remove the bucket policies

It would work and it takes the buckets offline for every legitimate consumer. Containing the resource rather than the identity has a much larger blast radius.

THE SIGNAL

Deactivating the key stopped new AssumeRole calls. The session already issued had up to an hour of life left.

An internet-facing web application on EC2 has a server-side request forgery flaw. An attacker uses it to reach the instance metadata service and retrieve the instance role's credentials, then uses them from an external IP.

The team contains the instance, revokes the role's sessions and patches the application flaw. The security architect is asked what change would prevent this class of attack succeeding again, even if another SSRF flaw is introduced.

What should the architect recommend?

- ☐ A Require IMDSv2 on all instances and set the metadata hop limit to 1
- ☐ B Remove the instance profile and put the credentials in environment variables instead
- ☐ C Add a WAF rule blocking requests containing the metadata service IP address
- ☐ D Enable GuardDuty Runtime Monitoring to detect metadata service access from the application

◆ PAUSE AND DECIDE

What should the architect recommend?

- A

 Require IMDSv2 and set the hop limit to 1
Correct. IMDSv2 needs a PUT to obtain a session token, which a simple SSRF cannot perform, and a hop limit of 1 stops the response leaving the instance.
- B

 Environment variables instead of an instance profile
Strictly worse. Long-lived static credentials that never rotate, readable by any process on the host and by the same SSRF flaw.
- C

 WAF rule blocking the metadata IP
Blocklisting one string in a class of attack with endless encodings. It also cannot see requests the application makes on the attacker's behalf.
- D

 GuardDuty Runtime Monitoring
Detective and worth having - it would tell you this happened. The question asks what prevents it, and detection is not prevention.

THE SIGNAL

'Even if another SSRF flaw is introduced' rules out fixing the flaw. Only a control that survives the next bug qualifies.

GuardDuty raises Policy:S3/BucketAnonymousAccessGranted. An engineer's automation has replaced a bucket policy on a bucket holding customer records with one granting s3:GetObject to Principal "*". The bucket has been publicly readable for around forty minutes. The security team must stop public access immediately across the whole account, before anyone works out which other buckets the same automation touched.

What is the FASTEST effective containment?

- ☐ A Delete the offending bucket policy and re-apply the previous version from source control
- ☐ B Apply an SCP denying s3:PutBucketPolicy across the organisation
- ☐ C Turn on S3 Object Lock in compliance mode for the affected bucket
- ☐ D Enable S3 Block Public Access at the account level

◆ PAUSE AND DECIDE

What is the FASTEST effective containment?

A Delete the policy and restore the previous version

Correct for this bucket, and the scenario asks for the whole account. It also leaves the automation free to do it again in ten minutes.

C Object Lock in compliance mode

Protects objects from deletion or modification. The problem is that strangers can read them, and Object Lock has nothing to say about reads.

B SCP denying s3:PutBucketPolicy

Prevents the next change and does nothing about the policy already in place. The bucket stays public while you congratulate yourself.

D Account-level S3 Block Public Access

Correct. One setting, immediate, and it overrides any bucket policy or ACL granting public access across every bucket in the account.

THE SIGNAL

'Across the whole account' and 'before anyone works out which other buckets' both point at the account-level control.

GuardDuty raises AttackSequence:EKS/CompromisedCluster. A container running a vulnerable web framework has been exploited, a cryptominer is running inside the pod, and the pod's service account token has been used to read Kubernetes secrets.

A platform engineer deletes the pod. Four minutes later, GuardDuty raises the same finding against a new pod on the same node.

Why did the finding recur, and what should the team do?

- ☐ A The node itself is compromised; terminate the node immediately to remove the miner
- ☐ B The Deployment's ReplicaSet recreated the pod from the same image; cordon the node and scale the Deployment to zero
- ☐ C GuardDuty replayed the archived finding; suppress the finding type and continue
- ☐ D The container image in ECR was re-pulled from a compromised registry; delete the ECR repository

◆ PAUSE AND DECIDE

Why did the finding recur, and what should the team do?

A Terminate the node

Same mistake as terminating an instance: it destroys memory and disk evidence, and the scheduler simply places the recreated pod on another node.

C GuardDuty replayed an archived finding

The scenario says a new pod. This is a new detection against a new resource, not a replay - and suppressing it would remove the signal from attack sequence correlation.

B The ReplicaSet recreated it; cordon the node and scale to zero

Correct. Deleting a pod managed by a Deployment guarantees a replacement. Cordoning stops new scheduling and scaling to zero stops the recreation.

D Delete the ECR repository

The image being vulnerable is the root cause and deleting the repository destroys the evidence and breaks every deployment using it.

THE SIGNAL

Killing a pod in Kubernetes is the same instinct as terminating an instance in an ASG. Both are supervised, and supervisors restart things.

CloudTrail shows the root user of a member account signing in from an unfamiliar country at 03:14, creating an IAM user with AdministratorAccess, and generating access keys for it. The organisation has an SCP on the account's OU denying several dangerous actions.

The security team is surprised, because they believed the SCP would have blocked this. The account was created five years ago and its root credentials have never been reviewed.

Which statement BEST explains the situation?

- ☐ A SCPs never apply to any root user, so no SCP could have stopped this
- ☐ B SCPs apply to member account root users, but the actions performed were not among those denied
- ☐ C The SCP failed because the account predates AWS Organizations SCP enforcement
- ☐ D Root user actions bypass CloudTrail, so the events shown must be from the IAM user

◆ PAUSE AND DECIDE

Which statement BEST explains the situation?

- A

 SCPs never apply to any root user
*Half-remembered and therefore dangerous. SCPs do not restrict the *management account* root. They do apply to member account root users.*
- B

 SCPs apply to member root; these actions were not denied
Correct. The SCP was in force and simply did not cover iam:CreateUser or iam:CreateAccessKey. An SCP restricts only what it names.
- C

 The account predates SCP enforcement
Fabricated. SCPs apply to any account in the OU regardless of when it was created. Age-based exemptions do not exist.
- D

 Root actions bypass CloudTrail
Root user activity is logged like any other principal, and monitoring it is a standard benchmark control precisely because it is visible.

THE SIGNAL

Two things to fix afterwards: broaden the SCP, and remove the root credentials entirely. Module 34 covers centralised root access.

A database administrator with legitimate, long-standing production access downloads the complete customer table over four evenings, using her normal role, from her normal laptop, during hours she often works. She resigns the following week.

No GuardDuty finding fires. No Config rule is breached. Every API call is authorised and succeeds. The security team only learns of it from a colleague.

What would MOST likely have detected this?

- A Amazon Macie automated sensitive data discovery on the S3 export bucket
- B An SCP denying database export actions outside a change window
- C IAM Access Analyzer external access findings on the production account
- D CloudTrail data events with a CloudWatch metric filter alarming on unusual export volume for that principal

◆ PAUSE AND DECIDE

What would MOST likely have detected this?

A Macie on the export bucket

Macie would confirm the exports contain personal data, which nobody doubted. It reports on content, not on who is reading how much of it.

C Access Analyzer external access findings

Access Analyzer finds resources reachable from outside the zone of trust. She was firmly inside it, using permissions deliberately granted to her.

B SCP denying exports outside a change window

Preventive, not detective, and the question asks what would have detected it. She also worked her normal hours, so a time window would not have caught her.

D Data events plus an alarm on unusual volume for that principal

Correct. Every call was authorised, so only volume and pattern distinguish this from her ordinary work. That needs data events, which are off by default.

THE SIGNAL

Authorisation controls cannot see an insider, because the insider is authorised. Only baselines and volume can.

Containment by resource type

COMPROMISE	CONTAIN BY
Leaked long-term key	Deactivate the key, then Deny on aws:TokenIssueTime for roles it assumed
EC2 instance credentials	Empty security group, detach from ASG, Deny on aws:TokenIssueTime
Public S3 bucket	Account-level Block Public Access, then repair the policy
Compromised EKS pod	Cordon the node, scale the Deployment to zero, rotate the secrets it read
Member account root user	Rotate and revoke, then remove root credentials centrally
Fully compromised account	Move it to a quarantine OU with a restrictive SCP

ON THE EXAM

One frame, six levers. Recognise the resource type and you already know which row you are in.

Five things to carry forward

- ◆ **Contain the identity, not the resource**, where you can. Removing a bucket policy stops the attacker and everyone else.
- ◆ **Supervised things restart**. Auto Scaling groups and ReplicaSets both replace what you delete.
- ◆ **SCPs do not restrict the management account root** – but they do apply to member account root users.
- ◆ **Detective is investigation, not detection**. Behavior graph, finding groups, one year of data, admin-only.
- ◆ **An insider is authorised**, so only volume and baseline catch them – which needs data events.

ON THE EXAM

That closes Domain 2. Next: Domain 3, infrastructure security, starting at the edge with CloudFront and AWS WAF.